

Lecture 9 Execution Control 3

- One of the fundamental aspects of a program is *execution control*.
- This lecture will introduce the concept of repeated execution of a block of code in a **loop** with the `while` conditional statement

In [2]:

```
import numpy as np
from numpy import random
from matplotlib import pyplot as plt
#
rng = random.default_rng(seed = 1121)
```

While Loops

- A `while` loop is a type of loop that runs as long as a *logical statement* is **True**.
 - When the logical condition becomes **False**, the code stops running.
 - The general form of a while loop in Python is below:
1. The while loop begins with a logical statement that tests a **variable** which initially returns be of value **True**
 2. There is an indented block of code inside the while loop.
 3. Within that code block there must be something that updates the value of the **variable**

```
while logical_statement:  
    do this  
    and that  
    and that  
    AND YOU MUST DO SOMETHING THAT POSSIBLY CHANGES  
    THE STATE OF THE LOGICAL STATEMENT TO FALSE
```

- Eventually, after one or more repetitions of the block of code, the logical statement returns a value **False** and the block of code no longer executes.

In [3]:

```
# Example 1
i = 0 #since i is the variable tested n the logical statement, it must be set to an initial value
while i<4: # this is the conditional statement which controls execution
    print(i)
    i = i+1 #this is a critical line, as it updates the value of i
```

0
1
2
3

CRITICAL STEP IN WHILE LOOPS

- There are three critical pieces to constructing a while loop.
 1. The variable(s) that will be used in the logical statement must be initialized to some value.
 2. A `while` statement performs execution control based on a logical statement that tests the variables being **True**.
 3. Inside the `while` loop, the variable(s) used in the logical statement must eventually be updated to a value that evaluates **False**
- When making use of a `while` loop it is critical that there always be a line inside the while loop that updates whatever is being tested by the logical statement.
- If that line is missing, the while loop will become an **infinite** loop and the code will only stop by an act of **VIOLENCE** by you against the code block, your VS Code instance, or in a worst case, against your computer (hard reboot).

Boolean Indicator Control

- Sometimes, its easy to think about this by using a Boolean Indicator variable to control the while loop.
- Here I rewrite the loop above with an indicator variable.

In [4]:

```
i = 0
keep_looping = True
while keep_looping:
    print(i)
    i = i+1
    if i >= 4:
        keep_looping = False
```

0
1
2
3

In [5]:

```
# Example 1
i = 0 #since i is the variable tested n the logical statement, it must be set to an initial value
while i<4: # this is the conditional statement which controls execution
    print(i)
    i = i+1 #this is a critical line, as it updates the value of i
```

0
1
2
3

Counters

- In many instances, we want to keep track of the number of times the block of code is executed by the loop using a counter.

Example Sum of Random Numbers to a Limit.

- This example shows a classical type of problem we often encountered in modeling and simulation.
- We want to execute some code until a variable reaches a critical value.

In [6]:

```
limit = 100 #set the limit
total = 0 #start at 0
nsamples = 0 #start a counter to keep track of the number of samples
while total < limit:
    sample = rng.integers(0,10) #get one random number from a multinomial distribution
    total = total + sample # this updates the value of total
    nsamples = nsamples + 1 # this keeps track of the number of samples taken
print('total = ',total)
print('nsamples = ',nsamples)
```

```
total = 101
nsamples = 21
```

- I could save a bit more information from this loop, by actually saving out the running values of total on each step.

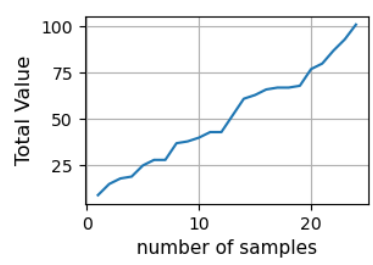
In [7]:

```
limit = 100 #set the limit
total = 0 #start at 0
totallist = list() # a list to keep track of the values on each iteration of the loop.
nsamples = 0 #start a counter to keep track of the number of samples
while total < limit:
    nsamples = nsamples + 1 # this keeps track of the number of samples taken
    sample = rng.integers(0,10) #get one random number from a normal distribution
    total = total + sample # this updates the value of total
    totallist.append(total) # I append the current value of total to the list.
print('total = ',total)
print('nsamples = ',nsamples)
print('totallist = ', totallist)
```

```
total = 101
nsamples = 24
totallist = [np.int64(9), np.int64(15), np.int64(18), np.int64(19), np.int64(25), np.int64(28), np.int64(28), np.int64(37), np.int64(38), np.int64(40), np.int64(43), np.int64(43), np.int64(52), np.int64(61), np.int64(63), np.int64(66), np.int64(67), np.int64(67), np.int64(68), np.int64(77), np.int64(80), np.int64(87), np.int64(93), np.int64(101)]
```


In [8]:

```
numberofsamples = np.arange(1,nsamples+1)
plt.figure(figsize = (3,2)) # I selected the figure dimension here
plt.plot(numberofsamples,totallist) # i use the quick an dirty way to make a plot here
plt.xlabel('number of samples',fontsize = 11)
plt.ylabel('Total Value',fontsize = 12)
plt.grid(True)
plt.show()
```



Example When will I have a million dollars?

- Suppose you have 20000 (or 2E04) dollars. You put in the bank, and each year they give you 5% interest (compounded annually).
- How many years will it take for you to have more than one million (1000000 or 1E06) dollars.

In [9]:

```
### What is the answer to the question? In terms of the code?
interest = 0.05 # interest rate
balance = 20000 # starting balance 2E04
nyears = 0
target = 1000000 # 1E06
while balance <= target: #Execute the block as long as my balance is less than or equal to target!
    balance = balance+interest*balance
    nyears = nyears+1
print('Balance of ', balance, 'after ', nyears, 'years')
```

Balance of 1040790.2624036912 after 81 years

Estimation Problems

- Estimation problems are useful for building intuition about the idea of measuring properties of human subjects.
- In an estimation problem, we are trying to approximate a quantity.

GEOMETRIC SERIES

$$geom = \sum_{n=1}^{\infty} \frac{1}{2^n} = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots$$

is a geometric series, that sums to 1 as **n** approaches infinity (∞).

In [10]:

```

N = 10
geomsumarray = np.zeros(N) #I want to save ever value in the loop.
geomsum = 0
for n in range(1,N+1): #notice the sum starts at 1 not at 0
    geomsum = geomsum + 1/2**n
    geomsumarray[n-1] = geomsum #note I had to index with n-1 because I made the loop start at 1.
error = 1-geomsum #I know the series sums to 1.
print(geomsum)
print('error is ', error)
print('geomsumarray is ', geomsumarray)
errorarray = 1-geomsumarray
print('errorarray is ', errorarray)

```

```

0.9990234375
error is 0.0009765625
geomsumarray is [0.5      0.75     0.875    0.9375    0.96875   0.984375
 0.9921875 0.99609375 0.99804688 0.99902344]
errorarray is [0.5      0.25     0.125    0.0625    0.03125   0.015625
 0.0078125 0.00390625 0.00195312 0.00097656]

```

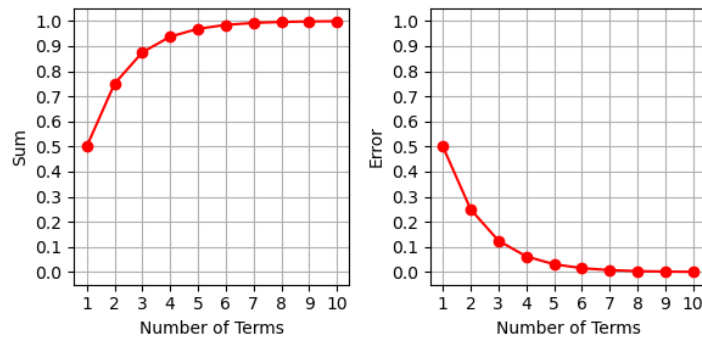
Subplots

Making multiple plots in the same graph

- Sometimes we want to put more than one graph in a figure. In this case, we can divide the figure into multiple plots.
- The syntax `plt.subplot(n,m,j)`:
 - tells python I want to divide the figure into n rows m columns.
 - the plot order is by row. j indicates where in the plot order you want to go.

In [11]:

```
numberofterms = np.arange(1,N+1) #I made a variable for my x axis
plt.figure(figsize = (6,3))
plt.subplot(1,2,1)
plt.plot(numberofterms,geomsumarray,'ro-')
plt.xlabel('Number of Terms')
plt.ylabel('Sum')
plt.ylim([-0.05,1.05])
plt.xticks(numberofterms)
plt.yticks(np.arange(0,1.1,0.1))
plt.grid()
plt.subplot(1,2,2)
plt.plot(numberofterms,errorarray,'ro-')
plt.xlabel('Number of Terms')
plt.ylabel('Error')
plt.ylim([-0.05,1.05])
plt.xticks(numberofterms)
plt.yticks(np.arange(0,1.1,0.1))
plt.grid()
plt.tight_layout() #this is CRITICAL with multiple plots
```



- I redid this with a slightly different indexing scheme

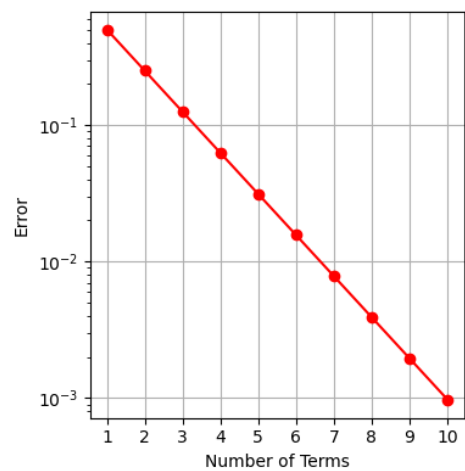
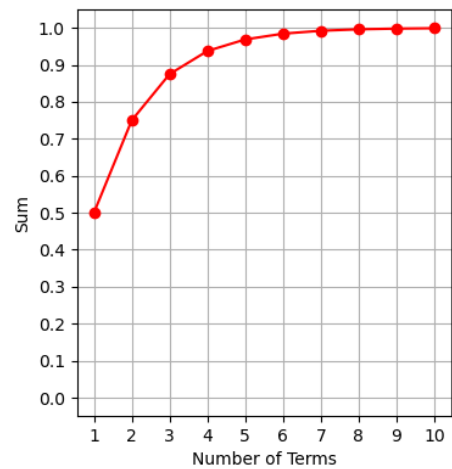
In [12]:

```
N = 10
geomsumarray = np.zeros(N) #I want to save ever value in the loop.
geomsum = 0
for n in range(N):
    geomsum = geomsum + 1/2**(n+1) #notice the sum starts with the exponent at 1 not at 0
    geomsumarray[n] = geomsum
error = 1-geomsum #I know the series sums to 1.
print(geomsum)
print('error is ', error)
print('geomsumarray is ', geomsumarray)
errorarray = 1-geomsumarray
print('errorarray is ', errorarray)
```

```
0.9990234375
error is 0.0009765625
geomsumarray is [0.5      0.75     0.875    0.9375    0.96875   0.984375
 0.9921875 0.99609375 0.99804688 0.99902344]
errorarray is [0.5      0.25     0.125    0.0625    0.03125   0.015625
 0.0078125 0.00390625 0.00195312 0.00097656]
```

In [14]:

```
numberofterms = np.arange(1,N+1) #I made a variable for my x axis
plt.figure(figsize = (4,8))
plt.subplot(2,1,1)
plt.plot(numberofterms,geomsumarray,'ro-')
plt.xlabel('Number of Terms')
plt.ylabel('Sum')
plt.ylim([-0.05,1.05])
plt.xticks(numberofterms)
plt.yticks(np.arange(0,1.1,0.1))
plt.grid()
plt.subplot(2,1,2)
plt.plot(numberofterms,errorarray,'ro-')
plt.xlabel('Number of Terms')
plt.ylabel('Error')
plt.yscale('log') #this is to make a log scale on an axis
plt.xticks(numberofterms)
plt.grid()
plt.tight_layout() #this is CRITICAL with multiple plots
```

When is my approximation good enough?

- In deciding how many terms we need we usually would have some threshold for our approximation to be *good enough*
- In many scientific libraries, this is an argument the function will take, called a tolerance.

In [15]:

```
tol = 0.01
#Lets compare our errorarray to the value of tol
print(errorarray < tol)
#Lets figure out how many terms that corresponds to
enoughterms = numberofterms[errorarray < tol] #I find where the errorarray < tol, then use that as an index into number of terms.
print(enoughterms)
#I want to know the least number of terms that would work.
minterms = enoughterms[0]
print('Number of terms: ', minterms)
```

```
[False False False False False False  True  True  True  True]
[ 7  8  9 10]
Number of terms:  7
```

- `np.where` returns indices but it has a slightly weird syntax

In [16]:

```
minterms = np.where(errorarray < tol)
print(minterms[0])
print(minterms[0][0])
```

```
[6 7 8 9]
6
```

- Or I could have done the whole thing as a `while` loop

In [17]:

```
tol = 0.01
n = 0
geomsum = 0 # the sum starts at 0
error = 1 # the error starts at 1
geomsum_list = list() # I want to keep track of the sums
while error > tol: #I want to keep adding terms as long as error is bigger than tolerance.
    n = n+1 #increment n
    geomsum = geomsum + 1/2**(n) #notice the sum starts with the exponent at 1 not at 0
    geomsum_list.append(geomsum) #append the current sum to the list
    error = 1-geomsum #I know the series sums to 1
geomsumarray = np.array(geomsum_list)
errorarray = 1-geomsumarray #compute the error at each term
print(geomsumarray)
print(errorarray)
print('Number of terms: ', n)
```

```
[0.5      0.75     0.875    0.9375    0.96875   0.984375  0.9921875]
[0.5      0.25     0.125    0.0625    0.03125   0.015625  0.0078125]
Number of terms:  7
```

Real-world Approximation Problems

- When we approximate something its usually because we ***don't know the correct answer***
- We are doing some process, e.g., collecting and averaging data, and we want to know if we can stop.
- Or if we are doing some data analysis thats complicated and incrementally getting closer to the correct answer we want to know when to stop
- So, we want to learn how to stop without knowing the correct answer.
- **The way we usually assess this is to examine if our answer is actually changing very much**
- In our gemetric series we can do this by asking if the new value we are adding on each stop is big enough or we can stop
- So we will compare the new value to be added to our tolerance

In [18]:

```

tol = 0.01
n = 0
geomsum = 0 # the sum starts at 0
sample = 1 # I make this larger than tol to start
geomsum_list = list() # I want to keep track of the sums
while sample > tol: #I want to keep adding terms as long as sample > tol
    # sample will be a term in the sequence.
    # notice I am NOT TESTING geomsum
    n = n+1 #Lets increment the index
    sample = 1/2**n #generate a new term
    #notice the sum starts with the exponent at 1 not at 0
    geomsum = geomsum + sample #add to the sum
    geomsum_list.append(geomsum) #append the current sum to the list
geomsumarray = np.array(geomsum_list)
errorarray = 1-geomsumarray #compute the error at each term
print(geomsumarray)
print(errorarray)
print('Number of terms: ', n)

```

```

[0.5      0.75      0.875     0.9375     0.96875    0.984375    0.9921875]
[0.5      0.25      0.125     0.0625     0.03125    0.015625    0.0078125]
Number of terms:  7

```

- Notice that in this implementation, I never made use of the correct value of the infinite sum (1)
- I am not comparing **geomsum** to the correct answer in order to decide when to stop.
- Instead, I asked the question, how much will my sum change if I add another term.
- the value of **sample** is how much the value will change.

Break and Continue Statements

- I really dont like this - but sometimes its unavoidable.
- `break` exits the loop if a condition is met
- `continue` skips the current iteration of the loop and proceeds to the next one.

Break Statement Syntax

```
for val in range_of_val:  
    some code 1  
    if condition:  
        break  
    some code 2
```

- My examples will use `for` loops. I cant think of any reason you would build this into a `while` loop but you can.
- note that some code 1 runs for every iteration of the loop until the loop exits at the break.
- note that some code 2 will not execute for the iteration where the loop breaks.

In [19]:

```
for i in range(5):  
    if i == 3:  
        break  
    print(i)
```

0
1
2

In [20]:

```
letters = ['a', 'b', 'c', 'd', 'e']  
for i in letters:  
    if i == 'c':  
        break  
    print(i)
```

a
b

Continue Statement Syntax

```
for val in range_of_val:  
    some code 1  
    if condition:  
        continue  
    some code 2
```

- Possibly you would build this into a `while` loop.
- note that some code 1 runs for every iteration of the loop until the loop exits at the break.
- note that some code 2 will not execute for each iteration where the `continue` executes.
- the idea is to skip over some value

In [21]:

```
for i in range(5):  
    if i == 3:  
        continue  
    print(i)
```

0
1
2
4

In [22]:

```
letters = ['a', 'b', 'c', 'd', 'e']  
for i in letters:  
    if i == 'c':  
        continue  
    print(i)
```

a
b
d
e